AWS Well-Architected Framework — Complete Notes

6 pillars · How AWS judges a "good" architecture

AWS Well-Architected | Pillars 6 | Type Framework | Free WA Tool

A reader-friendly, all-in-one reference for the AWS Well-Architected Framework.

Table of Contents

---

1. What is Well-Architected

The **AWS Well-Architected Framework** is AWS's set of **best practices** for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems in the cloud. It's **not a service** — it's a **lens** (a checklist + set of questions) used to evaluate whether an architecture is "good," and to find and fix risks before they bite.

> **Mental model:** Well-Architected is the **rubric AWS grades your architecture against.** Six pillars, each a set of design principles and questions. A "Well-Architected Review" walks the questions and surfaces risks.

Why it matters: it's a near-guaranteed conceptual interview question, and it gives you a vocabulary to justify design decisions ("this improves the *reliability* pillar via multi-AZ").

2. 📦 The 6 Pillars at a Glance

Pillar	One-line goal	Key services
Operational Excellence	Run & improve systems; automate; learn from failure	CloudFormation, CloudWatch, Systems Manager
Security	Protect data & systems; least privilege; defense in depth	IAM, KMS, WAF, CloudTrail, Shield
Reliability	Recover from failure; meet demand	Auto Scaling, Multi-AZ, Route 53, backups
Performance Efficiency	Use resources efficiently; pick the right tool	CloudFront, ElastiCache, right-sized instances, serverless
Cost Optimization	Avoid unnecessary spend; pay for what you use	Cost Explorer, Savings Plans, Spot, S3 lifecycle
Sustainability	Minimize environmental impact	Graviton, managed/serverless, right-sizing

> Memory aid (order): **"OPS-SEC-REL-PERF-COST-SUS"** — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. (Sustainability was added in 2021 — older material lists **5** pillars.)

3. ⚙️ Operational Excellence

Goal: run and monitor systems to deliver business value, and continuously improve.

- **Design principles:** perform operations **as code** (IaC), make **frequent, small, reversible** changes, refine procedures often, anticipate failure, learn from all operational events (blameless post-mortems).
- **In practice:** CloudFormation/Terraform, CI/CD pipelines, CloudWatch dashboards/alarms, runbooks & playbooks, Systems Manager for automation.

4. 🔐 Security

Goal: protect information, systems, and assets.

- **Design principles:** strong **identity foundation** (least privilege, central IAM), **traceability** (log & audit everything), **defense in depth** (multiple layers), **encrypt** in transit and at rest, keep people away from data (automate), prepare for incidents.
- **In practice:** IAM roles + MFA, KMS encryption, WAF + Shield, CloudTrail audit, GuardDuty/Security Hub, Secrets Manager, VPC isolation.

5. 🛟 Reliability

Goal: a workload performs its function correctly and recovers from failure.

- **Design principles: automatically recover** from failure, **test recovery** procedures, **scale horizontally** to increase availability, **stop guessing capacity** (elasticity), manage change through automation.
- **In practice:** Multi-AZ deployments, Auto Scaling, ELB health checks, Route 53 failover, backups + PITR, multi-region for DR, decoupling with SQS.

---

6. ⚡ Performance Efficiency

Goal: use computing resources efficiently as demand and tech change.

- **Design principles: democratize advanced tech** (use managed services), **go global in minutes**, use **serverless**, experiment more often, use the **right resource for the job** (mechanical sympathy).
- **In practice:** right-size instance types (incl. Graviton), caching (CloudFront, ElastiCache), serverless (Lambda, DynamoDB), read replicas, choosing the correct storage/database for the access pattern.

---

7. 💰 Cost Optimization

Goal: deliver value at the lowest price point.

- **Design principles:** adopt a **consumption model** (pay for what you use), measure **overall efficiency**, **stop spending on undifferentiated heavy lifting** (use managed services), analyze and **attribute spend**, right-size continuously.
- **In practice:** Savings Plans / Reserved Instances, **Spot** for interruptible work, right-sizing (Compute Optimizer), S3 lifecycle + Intelligent-Tiering, auto-scaling down, Cost Explorer + budgets + tagging.

---

8. 🌱 Sustainability

Goal: minimize the environmental impact of running cloud workloads.

- **Design principles:** understand your impact, set goals, **maximize utilization** (right-size, don't over-provision), adopt **efficient hardware/software** (e.g. **Graviton**), use **managed services** (shared, efficient), reduce downstream impact (less data movement, fewer devices needed).
- **In practice:** Graviton instances, serverless/managed services, efficient storage tiers, scaling to actual demand, region choice.

---

9. 🧩 Cross-Cutting Design Principles

Themes that appear across pillars:

- **Automate everything** (infra as code, recovery, scaling, ops).
- **Stop guessing capacity** — use elasticity instead of fixed sizing.
- **Test at production scale** and **practice failure** (game days, chaos).
- **Decouple** components for independent scaling & resilience.
- **Loosely couple + design for failure** — assume things break.
- **Data-driven decisions** — measure, then improve.

---

10. ⚖️ Trade-offs Between Pillars

Pillars often **pull against each other** — being "well-architected" means choosing consciously for the business need, not maxing every pillar:

- **Reliability vs Cost** — Multi-AZ/multi-region improves reliability but roughly doubles cost.
- **Performance vs Cost** — bigger instances / provisioned concurrency / DAX cost more.
- **Security vs Operational simplicity** — more controls add friction; automate to keep both.
- **Performance vs Sustainability** — over-provisioning wastes energy; right-size instead.

> 💡 Interviewers love: *"There's no single right architecture — it depends on the workload's priorities."* Name the trade-off you're making and why.

---

11. 🛠️ WA Tool, Lenses & Reviews

- **AWS Well-Architected Tool** — a **free** tool in the console to review a workload against the pillars, get a risk list (HRIs — High/Medium Risk Issues), and track improvements.
- **Lenses** — pillar questions tailored to a domain (Serverless, SaaS, Machine Learning, Data Analytics, IoT, Financial Services…).
- **Well-Architected Review** — the process of answering the questions with stakeholders to find and prioritize risks; repeat periodically and after major changes.

---

12. 🧠 Quick Mental Model

- **Well-Architected = AWS's rubric for a "good" cloud architecture** — a lens, not a service.
- **6 pillars:** Operational Excellence · Security · Reliability · Performance Efficiency · Cost Optimization · Sustainability. (Was 5 before Sustainability in 2021.)
- Each pillar = **design principles + questions + supporting services**.
- Pillars **trade off** (e.g. reliability vs cost) — choose deliberately for the workload.
- Recurring themes: **automate, stop guessing capacity, decouple, design for failure, measure**.
- Use the **free WA Tool** + **lenses** to run a **Well-Architected Review** and fix risks.

---

13. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about Well-Architected:

- **Q: Name the 6 pillars.** — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.
- **Q: Which pillar covers encryption and least privilege?** — Security.
- **Q: Which pillar covers Multi-AZ and Auto Scaling for recovery?** — Reliability.
- **Q: Which pillar covers Spot, Savings Plans, and right-sizing?** — Cost Optimization.
- **Q: What was the most recently added pillar?** — Sustainability (2021); before that there were 5.
- **Q: How do reliability and cost trade off?** — Multi-AZ/multi-region boosts reliability but increases cost; you balance per the workload's needs.

- **Q: Is Well-Architected a service?** — No — it's a framework/best-practice lens. The free **Well-Architected Tool** helps you review against it.
- **Q: What's a key principle shared across pillars?** — Automate (infra as code), stop guessing capacity (elasticity), and design for failure.
- **Q: Which pillar does "perform operations as code" belong to?** — Operational Excellence.
- **Q: How does Graviton relate to the pillars?** — It improves Cost Optimization, Performance Efficiency, and Sustainability (better price/performance per watt).
